

Evolución de la solución

Call Exercise - Class 2

De `proposedSolution` al diseño final

Agenda

1. Recap: *partial solution* (f8e06cc)
2. Reificación de PhoneNumber y Money
3. Requerimientos uno por uno
 - Flat fee
 - Floor en segundos facturados
 - Redondeo de minutos
 - Descuentos
4. Decorator + Factory sin romper la API
5. Lecciones: clean code, reificación y arquitectura

Recap: partial solution (f8e06cc)

```
public class Call {  
    private int originCountryCode, originAreaCode, originPhoneNumber,  
               destinationCountryCode, destinationAreaCode, destinationPhoneNumber,  
               durationInSeconds;  
    private CallCostCalculatorFactory factory;  
  
    public double calculateCost() {  
        return this.factory.createCalculator(this).calculateCost(this);  
    }  
}
```

- `call` con primitivos
- `CallCostCalculatorFactory` elige el primer calculator que aplica
- Cada calculator devuelve un `double` fijo
- **El costo no depende de la duración**

Problemas de la partial solution

- **Primitivos everywhere:** códigos de país, área y teléfono como `int` sueltos
- **Sin concepto de dinero:** usamos `double`, con problemas de precisión
- **Costo fijo:** `0.2`, `0.5`, `1.0` sin importar cuánto duró la llamada
- **Acoplamiento:** `call` sabe cómo comparar códigos para decidir si es local/nacional/internacional

Necesitamos reificar conceptos del dominio.

Reificación de PhoneNumber

```
@Value
public class PhoneNumber {
    int countryCode;
    int areaCode;
    int number;

    public boolean hasSameCountryCodeAs(PhoneNumber other) { ... }
    public boolean hasSameAreaCodeAs(PhoneNumber other) { ... }
}
```

call pasa a tener:

```
private PhoneNumber origin;
private PhoneNumber destination;
```

Y delega la comparación:

```
public boolean isNational() {
    return this.origin.hasSameCountryCodeAs(this.destination)
        && !this.origin.hasSameAreaCodeAs(this.destination);
}
```

Reificación de Money

```
public record Money(BigDecimal amount) {  
    private static final int SCALE = 2;  
    private static final RoundingMode ROUNDING_MODE = RoundingMode.HALF_UP;  
  
    public Money(BigDecimal amount) {  
        this.amount = amount.setScale(SCALE, ROUNDING_MODE);  
    }  
  
    public Money multiply(BigDecimal factor) { ... }  
    public Money add(Money other) { ... }  
}
```

- Escala y redondeo encapsulados
- Operaciones seguras sin errores de punto flotante
- El tipo comunica intención: esto es **dinero**

Impacto de reificar

- `Call.calculateCost()` devuelve `Money`
- `CallCostCalculator.calculateCost(Call)` devuelve `Money`
- Los tests comparan contra `BigDecimal` exacto

```
final var nationalCost = this.nationalCall.calculateCost();  
assertThat(nationalCost.amount()).isEqualTo(new BigDecimal("4.27"));
```

Menos `double`, más semántica de dominio.

Requerimiento 1: Flat fee

Ahora el costo depende de la duración y se suma una tarifa plana de conexión:

- **Local:** solo costo por minuto
- **Nacional:** costo por minuto + \$0.10 de conexión
- **Internacional:** costo por minuto + \$0.50 de conexión

Money necesita `add(...)` para componer el total.

Ejemplo completo: NationalCallCallCostCalculator

```
public class NationalCallCallCostCalculator implements CallCostCalculator {
    @Override
    public Money calculateCost(Call call) {
        final var ratePerMinute = new Money(new BigDecimal("0.50"));
        final var connectionFee = new Money(new BigDecimal("0.10"));
        final var minutes = minutesOf(call);
        return ratePerMinute.multiply(minutes).add(connectionFee);
    }

    private BigDecimal minutesOf(Call call) {
        return BigDecimal.valueOf(call.getDurationInSeconds())
            .divide(BigDecimal.valueOf(60), 10, RoundingMode.HALF_UP);
    }

    @Override
    public boolean applyFor(Call call) {
        return call.isNational();
    }
}
```

Requerimiento 2: Floor en segundos

Las llamadas nacionales menores a 30 segundos se facturan como 30 segundos.

```
private static final int MINIMUM_BILLING_SECONDS = 30;

private int billedDurationInSecondsOf(Call call) {
    return Math.max(call.getDurationInSeconds(), MINIMUM_BILLING_SECONDS);
}
```

Test:

```
final var shortNationalCall = new Call(..., 10, ...);
assertThat(shortNationalCall.calculateCost().amount())
    .isEqualTo(new BigDecimal("0.35"));
```

Requerimiento 3: Redondeo de minutos

Las llamadas internacionales redondean los minutos hacia arriba.

```
private BigDecimal billedMinutesOf(Call call) {  
    return BigDecimal.valueOf(call.getDurationInSeconds())  
        .divide(BigDecimal.valueOf(60), 0, RoundingMode.UP);  
}
```

Test:

```
final var shortInternationalCall = new Call(..., 61, ...);  
assertThat(shortInternationalCall.calculateCost().amount())  
    .isEqualTo(new BigDecimal("2.50"));
```

Renombramos `minutesOf` a `billedMinutesOf`: el nombre cuenta la historia.

Requerimiento 4: Descuentos

Nuevas reglas de negocio:

- **Off-peak:** 22:00 a 08:00 en días de semana → 60% del costo
- **Weekend:** sábados y domingos → 75% del costo

¿Cómo aplicarlos **sin ensuciar** cada calculator?

Patrón Decorator

```
public class DiscountingCallCostCalculator implements CallCostCalculator {  
    private final CallCostCalculator base;  
    private final DiscountPolicy discountPolicy;  
  
    @Override  
    public Money calculateCost(Call call) {  
        final var baseCost = this.base.calculateCost(call);  
        return this.discountPolicy.applyDiscounts(baseCost, call);  
    }  
  
    @Override  
    public boolean applyFor(Call call) {  
        return this.base.applyFor(call);  
    }  
}
```

- Envuelve a otro calculator
- Aplica descuentos sobre el costo base
- El calculator base **no sabe** que existe el descuento

Discount y DiscountPolicy

```
public interface Discount {  
    boolean appliesTo(Call call);  
    Money apply(Money cost, Call call);  
}  
  
public interface DiscountPolicy {  
    Money applyDiscounts(Money baseCost, Call call);  
}
```

- `SingleDiscountPolicy` : aplica un solo descuento
- `PrecedenceDiscountPolicy` : elige el primero que aplica (weekend antes que off-peak)

El problema con la API

Agregar descuentos obligó a cambiar el constructor de `CallCostCalculatorFactory`.

Eso **rompía** a todo el código que ya usaba la factory original.

¿Cómo agregamos una nueva funcionalidad sin forzar a los clientes a cambiar?

Solución: nueva factory sin romper la API

1. Convertimos `CallCostCalculatorFactory` en **interface**
2. `BaseCallCostCalculatorFactory` : comportamiento original
3. `DiscountingCallCostCalculatorFactory` : decora la base

```
public interface CallCostCalculatorFactory {  
    CallCostCalculator createCalculator(Call call);  
}
```

El cliente elige la composición que necesite.

BaseCallCostCalculatorFactory

```
@RequiredArgsConstructor
public class BaseCallCostCalculatorFactory implements CallCostCalculatorFactory {
    private final Collection<CallCostCalculator> calculators;

    @Override
    public CallCostCalculator createCalculator(Call call) {
        return this.calculators.stream()
            .filter(calculator -> calculator.applyFor(call))
            .findFirst()
            .orElseThrow(() -> new IllegalArgumentException(...));
    }
}
```

Mantiene la API y el comportamiento de la factory original.

DiscountingCallCostCalculatorFactory

```
@RequiredArgsConstructor
public class DiscountingCallCostCalculatorFactory implements CallCostCalculatorFactory {
    private final CallCostCalculatorFactory delegate;
    private final DiscountPolicy discountPolicy;

    @Override
    public CallCostCalculator createCalculator(Call call) {
        return new DiscountingCallCostCalculator(
            this.delegate.createCalculator(call),
            this.discountPolicy
        );
    }
}
```

Composición pura: toma cualquier factory y le agrega descuentos.

Uso final en Main

```
final CallCostCalculatorFactory factory = new DiscountingCallCostCalculatorFactory(  
    new BaseCallCostCalculatorFactory(List.of(  
        new InternationalCallCallCostCalculator(),  
        new NationalCallCallCostCalculator(),  
        new LocalCallCallCostCalculator()  
    )),  
    new PrecedenceDiscountPolicy(List.of(  
        new WeekendDiscount(),  
        new OffPeakDiscount()  
    ))  
);  
  
final Call call = new Call(..., factory);  
final var cost = call.calculateCost();
```

Lecciones de diseño

Clean code

- Nombres con intención: `billedDurationInSecondsOf`, `billedMinutesOf`
- Métodos privados pequeños, con un solo propósito
- Constantes con significado de negocio

Reificación

- `PhoneNumber` y `Money` como value objects del dominio
- El modelo refleja el lenguaje del negocio
- Menos primitivos, más semántica

Buena arquitectura

- Strategy + Factory + Decorator
- Cada clase tiene una razón para cambiar (SRP)
- Composición sobre herencia

El negocio desconoce el detalle

- `Call` solo ve la interface `CallCostCalculatorFactory`
- Los calculators no saben de descuentos
- Los discounts no saben de tarifas
- Todo se arma por composición afuera

Takeaways

1. **Reificar** primitivos relevantes del dominio (`PhoneNumber` , `Money`)
2. Usar **patrones** para agregar comportamiento sin tocar lo existente
3. **Proteger la API** cuando agregamos features: interfaces + composición
4. Cada regla de negocio debe tener un **test descriptivo**
5. El **dominio** no debe conocer los detalles de infraestructura o configuración

Anexo: resumen de commits

Commit	Cambio clave
f8e06cc	Partial solution: costos fijos con <code>double</code>
47b4313	Reificación de <code>PhoneNumber</code>
b99eae8	Reificación de <code>Money</code> y uso de duración
ca2f260	Flat fee + <code>Money.add(...)</code>
4b41fa0	Floor de 30s en llamadas nacionales
0f8910d	Redondeo hacia arriba en internacionales
85a87e8	Decorator + descuento off-peak
051696d	Nueva factory sin romper la API

¿Preguntas?

Gracias.